

Vrije Universiteit Amsterdam



Bachelor Thesis

How can the core stages of a 3D rendering pipeline be implemented and explained through a software renderer?

Author: *Kristóf Földházi (2771210)*

*1st Supervisor: Atze van der Ploeg
Daily Supervisor: Atze van der Ploeg
Second Reader: Thilo Kielmann*

*A thesis submitted in fulfillment of the requirements for
the VU Bachelor of Science degree in Computer Science*

2026.06.24

Abstract	3
1. Introduction	3
1.1 Research Question	3
1.2 Motivation	3
2. Background: A rendering pipeline	5
2.1: What is a software renderer?	5
2.2 What is a graphics rendering pipeline?	5
2.3 Coordinate spaces	7
2.4 Basic Mesh Representation	8
2.5 Triangle Rasterization	9
3. Renderer Design	10
3.1 Architecture	10
3.2 Main Components	12
3.3 Data Flow Through One Frame	14
3.4 Rendering modes	15
4. Implementation of the rendering pipeline	16
4.1 Mathematical and Implementation Conventions	16
4.1.1 Vector Convention	16
4.1.2 Matrix Order	17
4.1.3 Coordinate system	17
4.2 OBJ and texture loading	18
4.2.1 OBJ Loading	18
4.2.2 Texture Loading	20
4.3 Vector Operations	21
4.3.1 Vector Transformations	21
4.3.2 Dot Product	21
4.3.3 Cross Product and Surface Normals	22
4.3.4 Vector Length	22
4.3.5 Normalization	23
4.4 Camera and View Transformations	23
4.4.1 View Space	23
4.4.2 Camera In The Renderer	24
4.4.3 Camera Movement	24
4.4.5 Constructing The View Matrix	25
4.5 Backface Culling	27
4.6 Projection	30
4.6.1 The Projection Matrix	31
4.7 Scanline Triangle Rasterization	34
4.8 Texture Mapping	36
4.9 Flat Shading	42
4.10 Z-Buffer and Fog	43
5. Evaluation	45
6. Conclusion	45
References	47

Abstract

This thesis presents the design and implementation of a CPU-based 3D software renderer. The goal of this project is to explore, how to convert a 3D object into a 2D image without the use of a third party graphics API. The renderer supports loading OBJ models and texture data, transforming vertices through the rendering pipeline, camera movement, backface culling, projection, scanline triangle rasterization, Z-buffer-based visibility testing, texture mapping, flat shading and fog.

The thesis aims to explain the mathematical background of each step, supported with algorithmic examples from the implementation in Julia.

1. Introduction

1.1 Research Question

The main research question of the thesis is:

How can the core stages of a 3D rendering pipeline be implemented and explained through a software renderer?

To answer this question, the thesis will focus on the following subjects and subquestions:

- How can OBJ mesh data and texture data be represented internally for a software renderer?
- How can vertices be transformed from object space through view space and projection into screen-space coordinates?
- How can triangle rasterization, Z-buffering, texture mapping, flat shading and fog be implemented manually on the CPU?

1.2 Motivation

3D computer graphics is found everywhere in our modern life, and it plays a central part in many software systems, including video games, simulations, and engineering tools.

Modern rendering relies on the use of graphics APIs such as DirectX, OpenGL, or Vulkan, which hide most of the rendering math and logic behind their custom calls and functions and just return the desired result.

I have become very interested in the topic of 3D rendering, so to understand how a pixel gets drawn on the screen and how 3D objects become those pixels, I decided to explore how to build a CPU-based renderer from the ground up.

I approached the problem of 3D graphics from the beginning, and instead of using graphics APIs, this renderer performs the main stages of a rendering pipeline on the CPU and was implemented using the Julia programming language.

This thesis follows the same path as my renderer does, and goes through each phase of pixel calculation and transformation required to make a mesh come to life. This path starts with the reading and processing of model data, then goes through the transformation of an object with the use of a transformation pipeline and projection logic, then to rasterization, visibility, texturing, shading, and a final movable image output.

The purpose of this project is not to compete with any modern GPU-based system, but instead, to expose and explain each step of the rendering process. Because of the manual implementation, this thesis will demonstrate how a 3D model is converted from data to an image on screen.

2. Background: A rendering pipeline

2.1: What is a software renderer?

First, let's take a look at what a software renderer and a rendering pipeline are in general.

Generally speaking, a software renderer engine is a renderer that takes triangles in 3D space and projects them onto a 2D space that will be displayed as the output on the computer screen. It takes care of all the computation in software rather than relying on specific graphics hardware [1]. This, in turn, makes it less efficient for complex scenes, but it is quite useful for understanding the concepts used for the rendering process.

In the context of this thesis, the software renderer refers to a CPU-based implementation in Julia. It manually processes each input and decides what color to assign to each pixel.

2.2 What is a graphics rendering pipeline?

A graphics pipeline is the whole process by which we can convert the data of a 3D object into a 2D image, which will be viewed on the monitor. This process calculates the transformation between spaces and conversions from one type of data to another [2]. In modern graphics APIs such as OpenGL, this pipeline is usually handled by a combination of programmable shader stages and fixed-function hardware stages [3]. In a software renderer, in general, the same process can be implemented, but only using the CPU. This style of implementation will differ from a GPU-based pipeline, but the main conceptual stages will remain similar. First, the geometry data is loaded, transformed, projected, clipped, rasterized, depth-tested, and written into the frame buffer.

This thesis follows the same structure. The pipeline begins by loading the data from an OBJ file. An OBJ file contains the geometric data of the meshes that we wish to display. These contain vertex positions, face indices, texture coordinates, and sometimes normals. These coordinates are not yet ready to be displayed on screen; they need to go through several transformations in several coordinate spaces.

The first transformation is called the vertex transformation. A vertex usually begins in something called object space.

Its coordinates were defined in the 3D editor program, or were manually input as such. For this model to be placed in a scene, the coordinates have to be transformed

into world space. After that transformation, the scene needs to be transformed into view space. This space represents the perspective of the user/camera/eye. These transformations are commonly done with matrices called the object/model, view, and projection matrices. The model matrix is responsible for placing the object into the world, the view matrix is for the camera-related transformation, and the projection matrix takes care of turning the 3D coordinates into a 2D screen-mapped representation [4][5][6].

The implementation uses perspective projection, which just means that objects further from the camera appear smaller and objects closer to the camera appear bigger. This gives the rendered objects a sense of depth. This step is one of the most important ones that makes it possible to display a 3D scene on a flat display [4][5][6].

Before any triangles are drawn on the screen, they will be clipped, or culled away. The renderer performs backface culling and assumes that the remaining geometry is suitable for projection. Full frustum clipping is outside the current implementation scope. Culling, in this case backface culling, removes all the triangles, or “primitives,” that face away from the camera, thus reducing the amount of geometry sent for the later stages of the pipeline.

Once we have a final set of triangles that are ready to be displayed, the renderer has to decide whether to rasterize them or display them as wireframe. If rasterization is selected, the triangle only exists as a set of three coordinates, but the rasterization process converts these three points into a set of pixel positions that can be drawn on the screen [6]. For each of these final pixel positions, the renderer can calculate color, depth, texture coordinates, or lighting values.

Visibility is calculated and determined using the Z-Buffer. It is used to store and determine the depth of each pixel on the screen, thus enabling us to render the pixels in order (from closest to farthest) and, with this, making the objects appear in 3D. When a pixel is generated, it is then measured against the pixels at the same position, and if it is determined that it is closer than the other one, it is rendered [7].

Finally, the renderer calculates the color of visible pixels. Depending on the selected rendering mode, this may involve drawing only triangle edges in wireframe mode, filling triangles with a solid color, applying a texture using UV coordinates, applying flat shading based on the triangle normal, or blending the final color with a fog color based on depth. The resulting pixels are written to the framebuffer/colorbuffer, which represents the final image displayed to the user.

In this thesis, the pipeline is used both as a theoretical model and also as the structure of the implementation. Each stage of the pipeline corresponds to a Julia

code section. The following chapters will explain in detail each step both in theory, and they will also show how they were implemented in the final program.

2.3 Coordinate spaces

In a 3D rendering pipeline, the pixel coordinates do not immediately become screen coordinates. Screen coordinates would be the 2D coordinate system that represents each pixel location on the computer's monitor. Each coordinate space represents the same geometry, just from a different frame of reference. This allows the renderer to make calculations and decisions based on the shape of the model, its position in the world, and the camera's point of view [8].

Local Space is the coordinate system that the object was made in and is its own coordinate system. If an object was created, let's say in Blender, then most likely that object has an origin of (0,0,0), and that will not change, even if the cube ends up in a different location in a project [9] [23].

World Space coordinate system means that all of the imported objects get their individual coordinates in the world. These coordinates are transformed from local to world space with the use of the model matrix, and are relative to the world [9].

View Space can be seen as the camera of the program. View space coordinates are achieved by transforming the world space coordinates using the view matrix into the perspective of the camera's point of view. Meaning the coordinates from now on are "viewed" from where the camera is at a given time [9] [23].

Clip Space is used to determine which geometry lies inside the visible region of the camera. This region is often called the viewing frustum. If there is any geometry outside of this space, it should not appear in the final image. The coordinates from the view space are transformed by the projection matrix to enter clip space, also called homogeneous clip space coordinates. After the transform, there will be a perspective divide applied to the coordinates to transform them into normalized device coordinates (NDC) [9] [10] [23].

Normalized Device Coordinates (NDC) is achieved by dividing the `x`, `y`, and `z` components of a vertex by the homogeneous `w` component. In this implementation, and also in many other pipelines, the NDC coordinates are usually represented as normalized coordinates. This coordinate space is used to make a normalized range of values independent of screen resolution, so in the end, regardless of the actual monitor type or resolution, the images will have the same proportions [10] [23].

Screen Space is the final coordinate space and the final transformation that maps the previously normalized coordinates into a screen space coordinate system. This system uses pixel coordinates, so it has to map `x` between 0 and the screen width, and it has to map `y` between 0 and the screen height. The only catch is that the y coordinate most of the time has to be inverted, because in the screen space coordinate system, the `y` value increases downwards.

2.4 Basic Mesh Representation

Before any 3D object can be rendered, or rasterized, the renderer needs it in a form that it can process. In computer graphics, one of the common mesh representations is a polygon mesh. A mesh, in general, describes the surface of an object with the use of vertices, edges, and faces. The vertices define a point in space with `x`, `y`, (`z`) coordinates. An edge describes a connection between vertices, and a face describes the region enclosed by the edges [11] [12]

In this thesis, the most important type of face will be a triangle. A triangle is defined by three vertices and three edges, and together it forms a simple planar face. This type of face is suitable for rasterization, because the renderer can more easily determine which pixels lie inside the projected triangle. The renderer in this implementation will treat each model as a collection of triangles, rather than a continuous object.

A common vertex usually carries more information than just a position in 3D space. Taking the simplest vertex, it can be represented with three coordinates, usually named `x`, `y`, and `z`. However, rendering often requires more than just their location in 3D space, and more attributes are needed. These attributes can include texture coordinates, normal, or color values. In this renderer, the vertex position is important for placing and transforming the object in the world; the texture coordinates are used to map the loaded texture to the correct position; the color, normal, and surface directions are used to calculate shading and culling.

Texture (UV) coordinates are used to describe how a 2D texture is mapped onto a 3D surface. The 3D vertex describes a point location, while the UV coordinate describes where in the texture the color for that point can be found. During the rendering of a textured object, the UV coordinates are interpolated across the triangle, so the renderer can sample the correct color.

Normals describe the direction a surface is facing. They are important for calculating the lighting, shading, and visibility values [13]. This thesis implements flat shading, and in that case, a single normal can be used to determine the direction of a single triangle. This normal can be calculated from the triangle geometry by taking the

cross product of two triangle edges. The resulting vector is perpendicular to the triangle surface and can be compared with a light direction to calculate brightness.

This thesis implements the use of objects stored as OBJ files. In this type, the meshes are stored in an indexed representation. Instead of storing a complete copy of every vertex for every triangle, the mesh can store vertex data in lists and define faces by referencing elements from those lists. An OBJ file can contain different representations for vertex positions, texture coordinates, and normals [14]. This thesis utilizes this attribute of this representation by processing each line based on its starting letter.

For example, an OBJ face may contain entries in the form:

```
f 1/1/1 2/2/1 3/3/1
```

Each part of the face entry refers to a position index, a texture coordinate index, and a normal index [14].

In this project, mesh data loaded from OBJ files is converted into an internal triangle-based representation.

2.5 Triangle Rasterization

Mark Segal, Kurt Akeley, and Chris Frazier describes a rasterization process as: “Rasterization is the process by which a primitive is converted to a two dimensional image. Each point of this image contains such information as color and depth. Thus rasterizing a primitive consists of two parts. The first is to determine which squares of an integer grid in window coordinates are occupied by the primitive. The second is assigning a color and a depth value to each such square” [15].

For a basic understanding, a triangle rasterization can be seen as a coverage test. It checks for every pixel near the triangle whether it is inside or outside of the projected triangle. Usually, the centre of the pixel is checked against the triangle, and if the centre of the pixel is inside the triangle, then the whole pixel is considered to be inside [16].

In this thesis project, rasterization is implemented directly in Julia as part of the software rendering pipeline. After vertices are transformed, clipped, and projected into screen space, each triangle is passed to the rasterizer. The rasterizer determines which pixels belong to the triangle, calculates the required per-pixel values, and writes visible pixels to the framebuffer/colorbuffer.

3. Renderer Design

3.1 Architecture

The renderer is structured and designed around a simplified software rendering pipeline. In each frame, the camera updates, then processes the mesh through transformations, projection, rasterization, depth testing, and coloring. In this architecture, the loading of the assets happens once, at startup, and any other calculation happens once per frame. In this design, the asset loading, scene rasterization, camera control, render logic, and final output all have different, dedicated sections, so they can be understood independently.

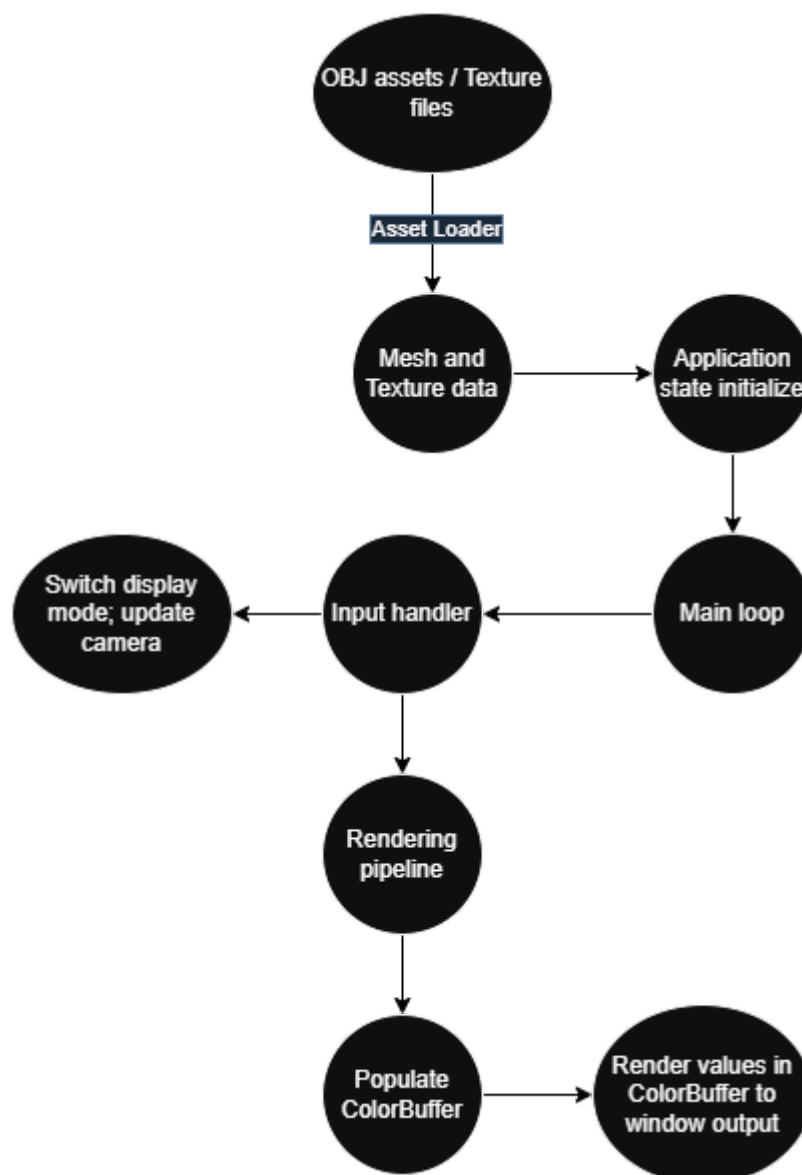


Figure 3.1 above shows a high-level overview of the implemented system architecture. The full implementation contains several lower-level steps, including frame timing, input handling, display-mode switching, matrix calculation, culling,

projection, rasterization, depth testing, and post-processing. However, to keep the architecture readable and understandable, this figure only shows the main data flow, omitting most steps and just focusing on the main ones. A more detailed control flow can be found in [38].

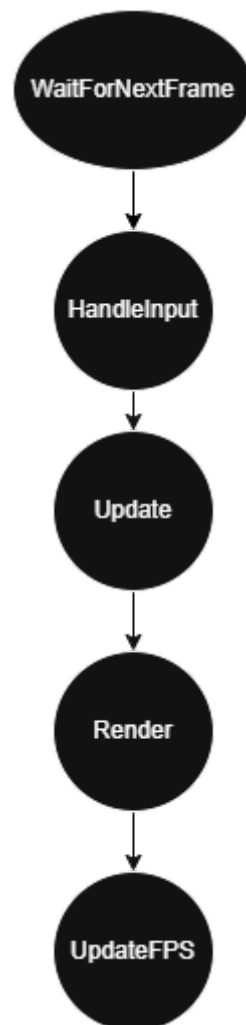


Figure 3.2 explains how each frame follows the same sequence. The renderer first controls the frame timing to ensure consistent speed, handles inputs from the mouse/keyboard that in turn will control the camera and the rendering modes, calculates and draws the frame into the ColorBuffer, and finally updates the FPS counter that gets displayed in the post-processing part.

3.2 Main Components

Figure 3.2.1 shows each component with its corresponding responsibility in the architecture. A mix of mutable and immutable structs was used to carry and store the values of each vertex, face, and complete mesh. The flow of how each value of a mesh is stored can be seen in Figure 3.2.2

COMPONENT	RESPONSIBILITY
AppState	It is a Mutable Struct, and it stores all the global variables and dependencies needed throughout the rendering of a frame.
Camera	It is also a Mutable Struct, and it stores the camera related vectors for the calculation process.
DisplayState	Used as an enum, to switch between rendering modes.
Mesh	Stores the values of a mesh, in terms of faces, vertices, scale, rotation, and translation in a Mutable Struct.
Face	Stores the three vector coordinates of a face, and the three UV coordinate values in a Mutable Struct.
Vec2, Vec3, Vec4	Vector type definitions, they store 2, 3, and 4 coordinates of a vector in an Immutable Struct.
Texture	Stores the width, height, and pixel values in an Immutable Struct.
Tex2	Type definition that stores the U and V coordinates of a texture.
Z-Buffer	Stores the closest depth values for each pixel.
ColorBuffer	Stores the final pixel color values before display.
OBJ loader	Read data from an OBJ file and converts the object data into mesh data for the renderer.
Light	Stores a Vec3 direction in a Mutable Struct.

Figure 3.2.1

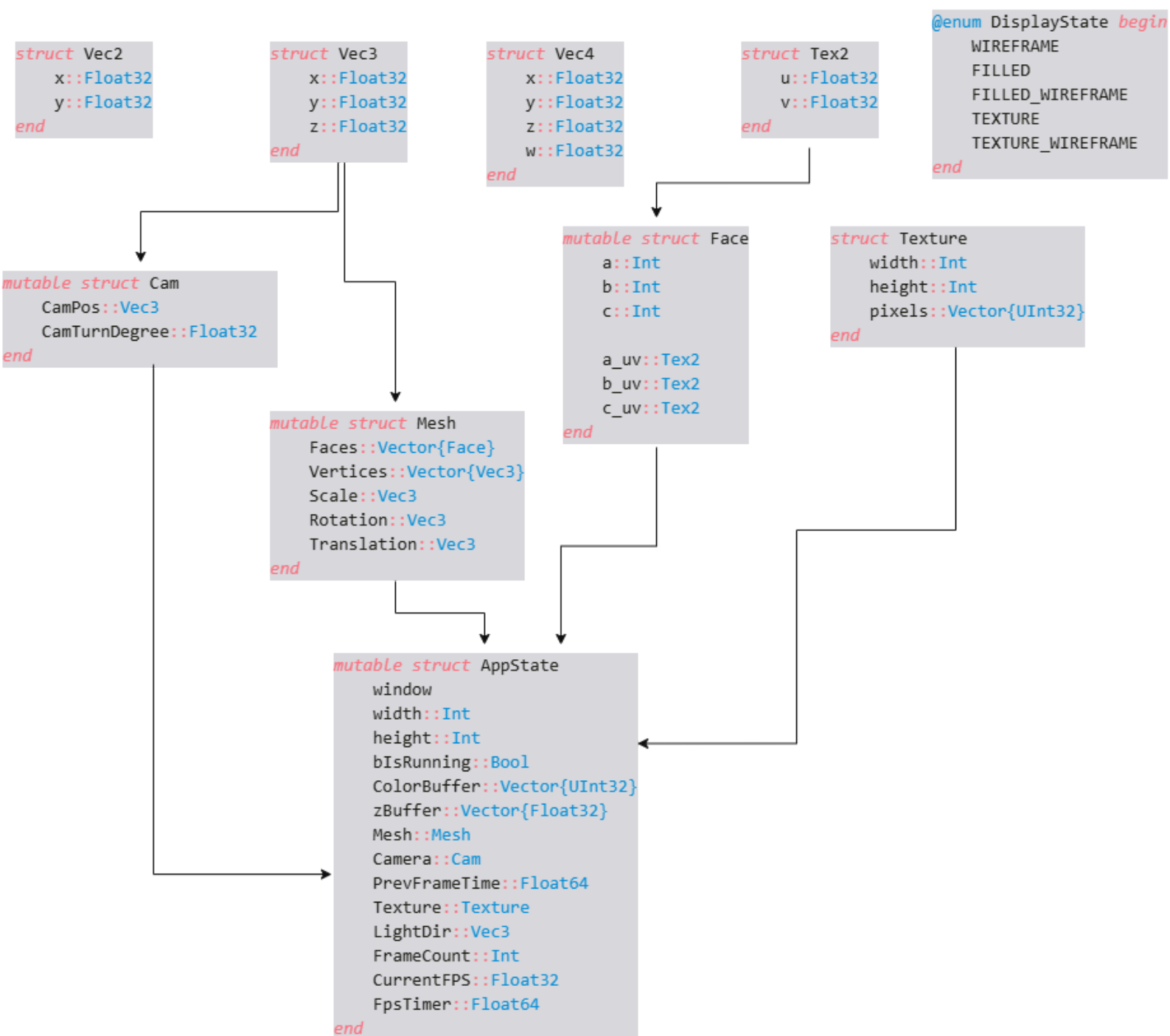


Figure 3.2.2

3.3 Data Flow Through One Frame

In each frame, the data follows the same general path and sequence of operations. The main purpose of this sequence is to update the state of the application, process the loaded mesh, and write the calculated values into the ColorBuffer for display.

At the beginning of each frame, the renderer handles frame timing. This is in place so that the program doesn't update faster than the selected maximum frame rate. This feature is in place so the object can be observed better, and so that it is consistent with transformation speed. After this, the renderer handles the user inputs. The user can move around 3D space by affecting the camera position and rotation, and the user can also switch between different rendering modes, like wireframe, filled, or textured representation.

After handling the inputs from the user and updating the necessary information for the next frame, the ColorBuffer gets cleared from the previous frame, so that the previous image doesn't remain visible. The ZBuffer also gets cleared to ensure that the depth comparisons will be performed with the data of the current frame.

The Update stage then prepares the data, vectors, and matrices necessary for rendering. The camera state is used to calculate the view transformations, and the mesh transformation values are used to create the calculations to place the mesh in the scene. After these preparations, the mesh is processed face by face. Each vertex in each face is transformed through the required coordinate spaces. At the end of the transformations, the visibility checks are performed, such as backface culling.

After it has been determined which geometry is visible, they are projected into screen space. After this step, the triangles/primitives are not only 3D geometric data. They have screen space position values that the rasterizer can use to calculate which pixels are covered by the projected triangle and how to color said pixels.

For each covered pixel, the renderer calculates the required per-pixel values, such as depth and texture coordinates. The interpolated depth value is compared with the current value stored in the ZBuffer. If the new pixel is closer than the previously stored value, the ZBuffer and ColorBuffer are updated. If the pixel is farther away, it is discarded.

When the ColorBuffer contains the necessary color values, the post processing affects are applied, such as fps count and fog. Finally, the contents of the ColorBuffer are passed to the window output function. This displays the completed frame on the screen. The next frame then begins with the same sequence.

With the main function being:

```
function main()
    app = InitWin()
    lastTime = time()

    while app.bIsRunning

        WaitForNextFrame!(app)
        currentTime = time()
        deltaTime = Float32(currentTime - lastTime)
        lastTime = currentTime

        HandleInput!(app, deltaTime)
        Update!(app)
        Render!(app)
        UpdateFPS!(app)
    end
end
```

3.4 Rendering modes

The renderer supports five different rendering modes. These modes are controlled with the use of the `DisplayState` enum, and it allows different visualization modes for the same model. The value of the enum is changed in the `HandleInput` function. Wireframe rendering mode makes the triangle structure of the mesh visible, which can be rendered alone or on top of the other rendering modes as well. It supports filled rendering, where the object is filled with a consistent color, and a texture mode, in which the model is displayed with the loaded texture (if any).

All of the rendering modes share most of the same pipeline. In every mode, the object is loaded from the OBJ file, stored, transformed, and projected. The difference between the modes appears after projection, when it is decided how to draw the projected triangle.

The different rendering modes are implemented as branches in the drawing stage of the pipeline. It is important that they are not separate systems, because they share most of the pipeline. This keeps the renderer simpler and readable.

In the Update loop it is:

```
if (CurrDisplayState == FILLED || CurrDisplayState == TEXTURE ||
    CurrDisplayState == FILLED_WIREFRAME ||
    CurrDisplayState == TEXTURE_WIREFRAME)

    DrawMeshFilled(app, TransformedVertices, FaceUVs,
CalculatedLightColor,
                    ProjectionMat)
end

if (CurrDisplayState == WIREFRAME || CurrDisplayState == FILLED_WIREFRAME ||
    CurrDisplayState == TEXTURE_WIREFRAME)

    DrawMeshWireframe!(app, TransformedVertices, 0x0000DD00,
ProjectionMat)
end
```

Where DrawMeshFilled draws filled objects, and DrawMeshWireframe draws objects just in wireframe mode.

4. Implementation of the rendering pipeline

4.1 Mathematical and Implementation Conventions

4.1.1 Vector Convention

In this thesis implementation, I use custom vector types to define geometric and texture data. These are named: Vec2, Vec3, Vec4, and Tex2. These types are used to make the mathematical operations in the pipeline explicit and easy to understand. Tex2 is essentially a Vec2, just with “u” and “v” values. It has been created only for readability purposes.

A Vec3 is a three-dimensional vector, with the coordinates named x, y, and z. This type is used both to define a point or a direction. A mathematical notation for a three-dimensional vector is:

$$v = (x, y, z)$$

And in Julia it is:

```
struct Vec3
    x::Float32
    y::Float32
    z::Float32
end
```

Vec2 is a two-dimensional vector, and it is rarely used, only towards the end of the pipeline for drawing calculations, and Vec4 is a four-dimensional vector. It has the coordinate values named x, y, z, and w. It is used when vertices need to be transformed by matrices, but that transformation will be discussed later. The w value in the four-dimensional vector is important because it is required for homogeneous coordinates. The use of such coordinates allows translation, rotation, scaling, and projection to be represented using matrix multiplication. After projection, w is also used for the perspective divide step.

The vectors use floating-point representation for their values, since the transformations, projections, interpolation, and lighting calculations often produce non-integer values.

4.1.2 Matrix Order

In this thesis, vertices are treated as column vectors, and thus mathematically a Vec4 is written as:

$$p = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

The order of operation, both in general matrix multiplication and in computer graphics calculations, matters a lot.

4.1.3 Coordinate system

The implementation uses a left-handed coordinate system. This means the z (depth value) grows inside the screen. This is the opposite in some graphics APIs, for example in OpenGL.

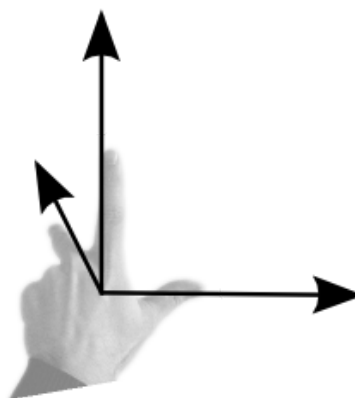


Figure 4.1: Coordinate system used in the graphics pipeline, from [17].

4.2 OBJ and texture loading

4.2.1 OBJ Loading

The OBJ file format (also known as the Wavefront OBJ file format) is one of the simplest and oldest file formats to store information about 3D geometry. It is considered simple because it doesn't support animations, and the description of material properties for each object is limited [14].

Since the file format is ASCII, it is relatively easy to read the values and reason about their meaning and role. The data is stored as a series of lines, with each having a certain starting letter(s) such as:

- “v” -- a line starting with this indicates that a vertex definition will follow with three floats [14]. Example: (v -0.550532 -0.462150 -7.045138)
- “vn” -- this means that a normal definition will follow with three floats [14]. Example: (vn -0.0000 1.0000 -0.0000)
- “vt” -- will indicate texture u, v coordinates in the form of two floats [14]. Example: (vt 0.625000 0.000000)
- “f” -- defines a face of the mesh [14]. Example: (f 2/1/1 3/2/1 1/3/1)

“v”, “vn” and “vt” are self explanatory, however “f” will need some more explanation.

The face definitions can come in more than one form. In this project, and also in the example above, I use a triangulated mesh, meaning each face has three times three values. If it has four times three values,

In this project, the face definition will carry several pieces of information with it. It will be decoded as such: (f v1/vt1/vn1 v2/vt2/vn2 v3/vt3/vn3). This way, the face defines both vertex, texture coordinate, and normal indices.

The OBJHandle looks through each line, parses the vertex and vertex coordinate values, and stores them. The normal values are not used in this implementation; they will be later calculated from the triangles for backface culling and flat shading. The loader supports triangulated faces and faces with more than three vertices. Faces that have more than three vertices will be triangulated by using a fan

triangulation method. This means that the first vertex will be an anchor, and the next indices are paired up, and the three of them together will form a triangle. Using this method, a quad would be converted into two triangles. The loader is called during the initialization process.

```
function LoadMeshFromOBJ(filename::String)::Mesh
    vertices = Vec3[]
    faces = Face[]
    texCoords = Tex2[]

    for line in eachline(filename)
        line = strip(line)

        if isempty(line) || startswith(line, "#")
            continue
        end

        parts = split(line)

        if parts[1] == "v"
            x = parse(Float32, parts[2])
            y = parse(Float32, parts[3])
            z = parse(Float32, parts[4])
            push!(vertices, Vec3(x, y, z))
        elseif parts[1] == "vt"
            u = parse(Float32, parts[2])
            v = parse(Float32, parts[3])
            push!(texCoords, Tex2(u, v))
        elseif parts[1] == "f"
            faceTokens = parts[2:end]
            parsed = [ParseOBJFaceToken(token) for token in faceTokens]

            for i in 2:length(parsed)-1
                vi1, ti1, ni1 = parsed[1]
                vi2, ti2, ni2 = parsed[i]
                vi3, ti3, ni3 = parsed[i+1]
                face = Face(
                    vi1,
                    vi2,
                    vi3,
                    GetUV(texCoords, ti1),
                    GetUV(texCoords, ti2),
                    GetUV(texCoords, ti3)
                )
                push!(faces, face)
            end
        end
    end
end
```

```

end
return Mesh(
    faces,
    vertices,
    Vec3(1.0f0, 1.0f0, 1.0f0),
    Vec3(0.0f0, 0.0f0, 0.0f0),
    Vec3(0.0f0, 0.0f0, 5.0f0)
)
end

```

4.2.2 Texture Loading

This implementation only supports the loading of textures in PNG encoding, with the use of the following function:

```

function LoadTextureFromPNG(filename::String)::Texture
    img = load(filename)
    height, width = size(img)
    pixels = Vector{UInt32}(undef, width * height)

    for y in 1:height
        for x in 1:width
            c = RGBA{Float32}(img[y, x])
            r = round(UInt8, clamp(c.r, 0.0f0, 1.0f0) * 255.0f0)
            g = round(UInt8, clamp(c.g, 0.0f0, 1.0f0) * 255.0f0)
            b = round(UInt8, clamp(c.b, 0.0f0, 1.0f0) * 255.0f0)
            a = round(UInt8, clamp(c.alpha, 0.0f0, 1.0f0) * 255.0f0)
            index = (y - 1) * width + x
            pixels[index] = PackRGBA(r, g, b, a)
        end
    end
    return Texture(width, height, pixels)
end

function PackRGBA(r::UInt8, g::UInt8, b::UInt8, a::UInt8)::UInt32
    return (UInt32(a) << 24) |
           (UInt32(r) << 16) |
           (UInt32(g) << 8) |
           UInt32(b)
end

```

The loader PackRGBA uses a bitwise OR operation to make a 32-bit integer from four 8-bit channels. This exposes the first limitation with this approach, since it cannot load anything other than a PNG with 8-bit depth values.

The PNG gets converted into 0xAARRGGBB format, which is popular with rendering pipelines.

4.3 Vector Operations

4.3.1 Vector Transformations

Vector transformation in 3D graphics is an essential component of the rendering pipeline, since this part handles the scaling, rotation, and translation of the selected vertices, which in turn transforms the faces of the mesh.

Each operation requires a different matrix multiplication, but if we chain those together, we get something that's called a World Matrix. That is achieved by multiplying the scale, rotate, and translate matrices together before the vector operation [22] [23].

$$M = T * R * S$$

Where M is the resulting combined matrix, S is the scale matrix, R is the rotation matrix, and T is the translation matrix. Since matrix multiplication is not commutative, the order in this operation matters a lot [22].

A vertex in object space then can be transferred into world space by [22]:

$$p_{world} = M * p_{object}$$

After this operation, view and projection are applied by:

$$p_{clip} = P * V * M * p_{object}$$

where V is the view matrix, and P is the projection matrix. This chained multiplication expresses the path of a vertex through the transformation pipeline [24].

4.3.2 Dot Product

The dot product of two vectors is used to determine the alignment of two vectors. For two vectors, this operation can be written as:

$$a \cdot b = a_x * b_x + a_y * b_y + a_z * b_z$$

```
function Vec3Dot(a::Vec3, b::Vec3)::Float32
    return Float32(a.x * b.x + a.y * b.y + a.z * b.z)
end
```

If both values are normalized, a final number close to 1 means that the two vectors are pointing in a similar direction; a value closer to 0 means the vectors are perpendicular, and a value close to -1 means they are pointing in the opposite direction.

4.3.3 Cross Product and Surface Normals

Calculating the cross product of two vectors returns the normal vector (N) of those two vectors.

$$N_x = a_y * b_z - a_z * b_y$$

$$N_y = a_z * b_x - a_x * b_z$$

$$N_z = a_x * b_y - a_y * b_x$$

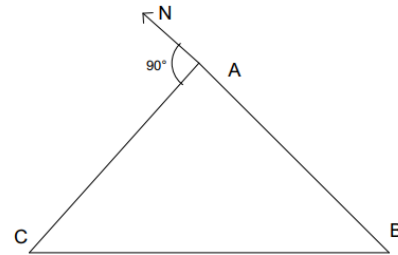


Figure 4.3.3

Figure 4.3.3 shows the resulting normal vector from the equation:

$$(B - A) \times (C - A)$$

Using this equation, the order matters, since:

$$(B - A) \times (C - A) \neq (C - A) \times (B - A)$$

The cross product of two vectors also gives the area of the parallelogram formed by those vectors, shown in Figure 4.3.4. [21].

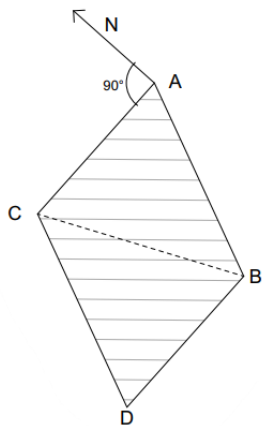


Figure 4.3.4

4.3.4 Vector Length

The magnitude/length of a 2D vector can be calculated[25]:

$$\|\vec{v}\| = \sqrt{(v.x)^2 + (v.y)^2}$$

```
function Vec2Len(a::Vec2)
    return sqrt(a.x * a.x + a.y * a.y)
end
```

and for 3D vectors:

$$\|\vec{v}\| = \sqrt{(v.x)^2 + (v.y)^2 + (v.z)^2}$$

```
function Vec3Len(a::Vec3)
    return sqrt(Vec3Dot(a, a))
end
```

4.3.5 Normalization

Normalising a vector converts it into a unit vector, meaning a vector with a length of 1.

Importantly, it keeps its direction, but removes the magnitude.

```
function Normalize(v::Vec3)
    len = Vec3Len(v)
    if len == 0
        return Vec3(0, 0, 0)
    end
    return v / len
end
```

This operation is used when comparing vector directions. In that case, magnitude is not a factor.

4.4 Camera and View Transformations

After a model has been processed and loaded into the scene, it still needs to be determined from which direction it should be viewed. Before implementing a camera, the “eye” position has always been at the centre. This will have to change if we want to move the camera around the scene. In fact, the camera is not the one that will move, but instead, the object itself will have some transformations applied to it to mimic camera movement.

Before projecting the object, it will need to be transformed relative to the moving “eye” point. These transformations will be mathematically reversed, since if the camera moves to the left, the world has to move to the right.

4.4.1 View Space

For the camera transformations to take place, a new coordinate space needs to be created. This system is called the view space. So far, the object started in object space from the 3D software it was created in; after that, it got transformed into world space, which placed the object into the world with the desired rotation, scale, and position. Now it is time to transform that object into view space with the help of more matrix multiplications.

4.4.2 Camera In The Renderer

The renderer uses a custom mutable camera struct that stores the position and turn degree in radians as such:

```
mutable struct Cam
    CamPos::Vec3
    CamTurnDegree::Float32
end
```

The camera direction is calculated from:

```
function GetCameraDirection(app::AppState)::Vec3
    return Normalize(Vec3(
        sin(app.Camera.CamTurnDegree), #line 3
        0.0f0,                          #line 4
        cos(app.Camera.CamTurnDegree)  #line 5
    ))
end
```

Line 3 calculates the x coordinate, line 4 is the y coordinate, which is set to 0 since the renderer doesn't implement pitch, and line 5 is the z coordinate.

4.4.3 Camera Movement

W/S moves the camera forward and backwards with the use of:

```
forward = GetCameraDirection(app)

if key_down(app.window, MiniFB.KB_KEY_W)
    app.Camera.CamPos = app.Camera.CamPos + forward * (moveSpeed *
deltaTime)
end

if key_down(app.window, MiniFB.KB_KEY_S)
```

```

        app.Camera.CamPos = app.Camera.CamPos - forward * (moveSpeed *
deltaTime)
    end

```

With the arrow keys, the camera can be moved up/down and turned left/right as such:

```

if key_down(app.window, MiniFB.KB_KEY_UP)
    app.Camera.CamPos = Vec3(
        app.Camera.CamPos.x,
        app.Camera.CamPos.y + moveSpeed * deltaTime,
        app.Camera.CamPos.z
    )
end

if key_down(app.window, MiniFB.KB_KEY_DOWN)
    app.Camera.CamPos = Vec3(
        app.Camera.CamPos.x,
        app.Camera.CamPos.y - moveSpeed * deltaTime,
        app.Camera.CamPos.z
    )
end

if key_down(app.window, MiniFB.KB_KEY_RIGHT)
    app.Camera.CamTurnDegree += turnSpeed * deltaTime
end

if key_down(app.window, MiniFB.KB_KEY_LEFT)
    app.Camera.CamTurnDegree -= turnSpeed * deltaTime
end

```

where deltaTime is used to keep the movement consistent regardless of frames per second. moveSpeed and turnSpeed are predefined values.

4.4.5 Constructing The View Matrix

```

function LookAtItem(eye::Vec3, target::Vec3, up::Vec3)::Matrix{Float32}

    z = target - eye #line 3
    z = Normalize(z) #line 4
    x = Cross(up, z) #line 5
    x = Normalize(x) #line 6
    y = Cross(z, x) #line 7

```

```

viewMatrix = Float32[
    x.x x.y x.z -Vec3Dot(x, eye)
    y.x y.y y.z -Vec3Dot(y, eye)
    z.x z.y z.z -Vec3Dot(z, eye)
    0.0f0 0.0f0 0.0f0 1.0f0
]

return viewMatrix
end

```

This is the most crucial part of the camera movement, since with the use of the output of this function, the object will be transformed into view space.

In the implementation, the view matrix is constructed using a look-at function:

```
ViewMat = LookAtItem(app.Camera.CamPos, LookTarget, Up)
```

The function receives three vectors:

```
function LookAtItem(eye::Vec3, target::Vec3, up::Vec3)::Matrix{Float32}
```

Here, “eye” is the camera position in world space, “target” is the point the camera is looking at, and “up” is a reference vector that defines which direction should be considered upward. The output of the function is a matrix that transforms world-space coordinates into camera-space coordinates.

Lines 3 and 4 calculate the camera's forward direction. The resulting vector z points from the camera position to the target. z is a unit vector, since we are only interested in its direction, and it is pointing in the positive direction in this implementation.

Lines 5 and 6 calculate the x direction. In other words, the right direction from the camera origin. Since we know the up and z direction, and both are unit vectors, the right direction will be perpendicular to these two; thus, the cross product is used. This x-axis is local to the camera.

On line 7, the up direction is calculated as a precaution. The up direction is passed through the function; however, I wanted to make sure that the three vectors of the camera are orthogonal to each other.

After this, the camera has its own local coordinate system with x, y, and z.

The view matrix is then constructed by translating the scene inversely and rotating it with reverse orientation. This implies that the rotation matrix, which has the form [26] [27]:

$$M_r = \begin{bmatrix} x_x & y_x & z_x & 0 \\ x_y & y_y & z_y & 0 \\ x_z & y_z & z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

has to be inverted. But since the rotation matrix is orthogonal, it can be inverted by taking the transpose of the original matrix.

$$M_r = \begin{bmatrix} x_x & x_y & x_z & 0 \\ y_x & y_y & y_z & 0 \\ z_x & z_y & z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This transpose of the original rotation matrix multiplied by the translation matrix, which is [26][27]:

$$M_t = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad M_t = \begin{bmatrix} 1 & 0 & 0 & -eye_x \\ 0 & 1 & 0 & -eye_y \\ 0 & 0 & 1 & -eye_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The equation comes out to be:

$$M_{view} = M_r * M_t$$

Resulting in the following view matrix:

$$M_{view} = \begin{bmatrix} x_x & x_y & x_z & -dot(x, eye) \\ y_x & y_y & y_z & -dot(y, eye) \\ z_x & z_y & z_z & -dot(z, eye) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

After obtaining this matrix as the output of the LookAtItem function, it is used to transform each vertex of each face, thereby producing camera/view-space coordinates [26] [27].

4.5 Backface Culling

In 3D rendering pipelines, backface culling is a common visibility optimization technique that, in simple terms, removes the triangles facing away from the camera [18]. This also works if the camera is not static. In a 3D model, most of the time, there are several triangles (depending on orientation and construction of the model) that are facing away from the camera, or better yet, the user. Drawing these triangles would not necessarily contribute positively to the final image, since it causes cluttering, and it is harder to see the object in 3D [18] [28]; however, in this implementation, this can be turned on and off during rendering, for testing purposes. Backface culling rejects these triangles from the pipeline before rasterization, depth testing, or projection calculations would take place on them.

In this renderer, backface culling is performed in view space. This is the case because we want to have backface culling relative to the moving camera, and for that, we need to work in view space, as discussed before.

The general idea here is to compare the direction the triangle is facing with the direction of the camera to the triangle.

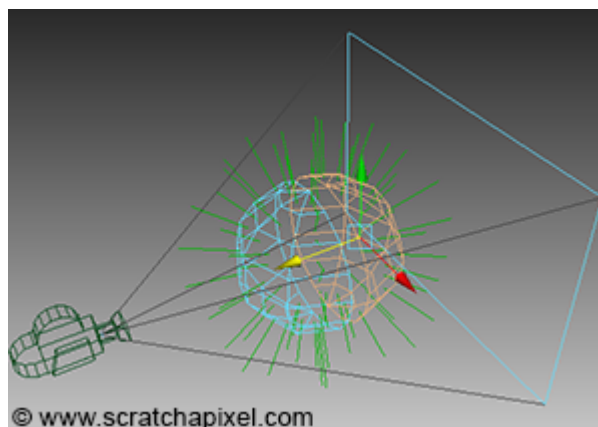


Figure 4.5 From [28]

For this calculation, the cross product will be of help to us as such:

To calculate the cross product and derive the normal vector of the triangle face, we need two edges [28]:

$$e1 = v1 - v0$$

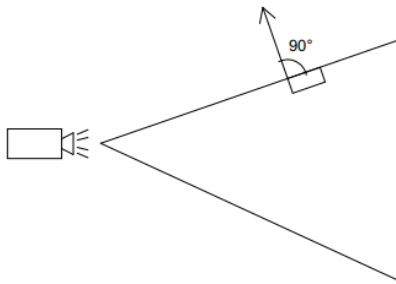
$$e2 = v2 - v0$$

where $e1$ and $e2$ are the resulting edges, and $v0$, $v1$, $v2$ are the three vectors that make up the triangle.

```
edge1 = v1 - v0
edge2 = v2 - v0
```

The surface normal is then calculated:

$$n = \text{normalize}(e1 \times e2)$$



where n denotes the resulting normal unit vector, and $e1 \times e2$ returns the cross product. In the implementation:

```
normal = Normalize(Cross(edge1, edge2))
```

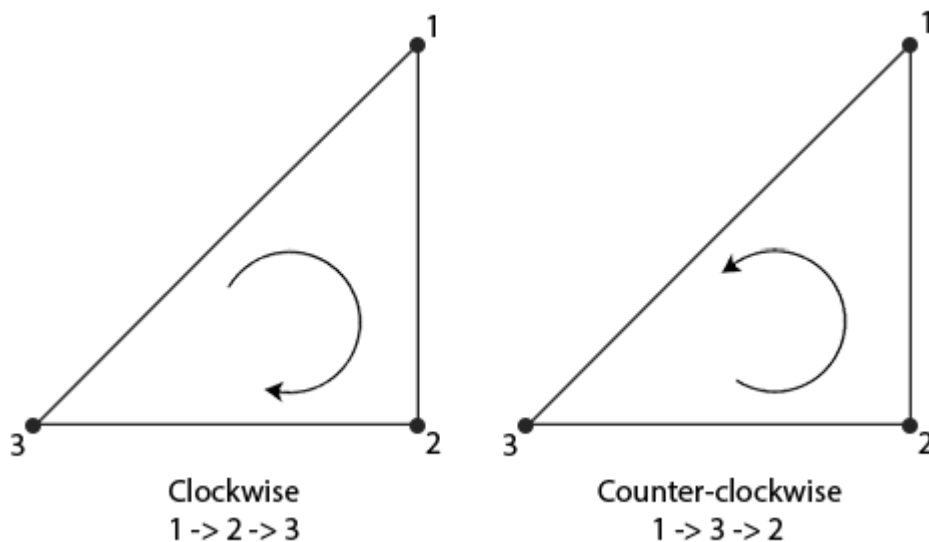


Figure 4.5 shows the two different orderings. From [18].

The implementation assumes a clockwise winding order for every triangle vertex.

Next, the renderer calculates the vector from the camera to one point of the triangle as such:

$$\text{CameraVector} = \text{Normalize}(v0 - \text{CamPos})$$

where $v0$ is a vertex on the triangle. Since this calculation takes place in view space, the camera position should always be at the origin (0,0,0). In the implementation:

```
cameraToTriangle = Normalize(v0 - Vec3(0.0f0, 0.0f0, 0.0f0))
```

After, the the dot product of the two vectors is calculated:

$$d = \text{dot}(\text{normal}, \text{CameraVector})$$

which in the implementation is:

```
dotProd = Vec3Dot(normal, cameraToTriangle)
```

The dot product indicates whether the normal points generally toward or away from the camera. If the normal and the view vector point in a similar direction, the triangle is facing away from the camera and can be skipped. If they point in opposite directions, the front side of the triangle is visible, and the triangle should continue through the pipeline.

As a last step, this comparison returns a boolean value to the renderer:

```
return dotProd >= 0.0f0
```

With the full implementation of the culling logic being:

```
function IsCulled(vertices::Vector{Vec3})::Bool
    v0 = vertices[1]
    v1 = vertices[2]
    v2 = vertices[3]

    edge1 = v1 - v0
    edge2 = v2 - v0

    normal = Normalize(Cross(edge1, edge2))

    cameraToTriangle = Normalize(v0 - Vec3(0.0f0, 0.0f0, 0.0f0))
    dotProd = Vec3Dot(normal, cameraToTriangle)

    return dotProd >= 0.0f0
end
```

Backface culling does not solve the problem of rejecting faces that are behind others; instead, it rejects faces that face away from the camera's point of view.

4.6 Projection

When talking about 3D graphics, there are several projection approaches one can take, but this thesis will focus on Perspective Projection.

In perspective projection, using a left-handed coordinate system, the larger the z-value, the closer the object is to the camera.

After the view transformation, each vertex is expressed relative to the camera. However, the framebuffer is two-dimensional, so the renderer still needs to convert the 3D view-space position into a 2D screen-space position. This is the problem that projection aims to solve

Perspective projection is based on dividing the horizontal and vertical coordinates of a point by its depth. If two points have the same x coordinate but one point is farther away, the farther point should appear closer to the center of the screen. This is achieved by division by z. This part is called perspective divide.

The pipeline for this section goes as follows:

First, we take the view space coordinates produced from before, and the end goal is to have screen-space coordinates, ready to render.

The journey there, however, requires a few more steps.

The first step is to make a projection matrix, transform the coordinates into clip space, then into NDC, and finally into screen space.

4.6.1 The Projection Matrix

To construct the projection matrix, we need a few values to calculate first.

The first one will be the aspect ratio, which describes the proportional relationship between the width and height of the screen and can be calculated by:

$$a = w / h$$

where h is the height of the screen, and w is the width.

The field of view controls how wide the virtual camera lens is. A smaller FOV creates a more zoomed-in image, while a larger FOV creates a wider view. The focal scaling factor is calculated as:

$$f = 1 / \tan(\alpha/2)$$

where α is the vertical field-of-view angle.

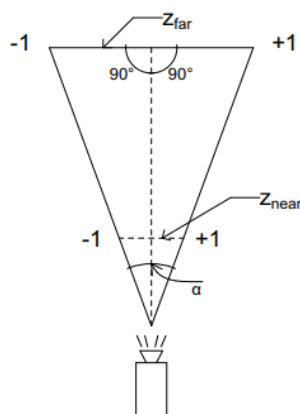


Figure 4.6.1

Additionally, near and far clipping-plane distances are used. In this renderer, znear is set to 0.1, and zfar is set to 100.0.

```

ProjectionMat = MakePerspective(
    Float32(pi / 3), #line2
    aspect,          #line3
    0.1f0,           #line4
    100.0f0          #line5
)

```

Line 2 defines the FOV angle, in this case 60 degrees. Line 3 passes the aspect ratio of the screen, Line 4 defines znear, and Line 5 defines zfar.

With these values, the basic perspective projection matrix becomes:

$$\begin{bmatrix} f/a & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{z_{far}}{z_{far}-z_{near}} & -\frac{z_{far} * z_{near}}{z_{far}-z_{near}} \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

The value 1 at row 4, column 3 causes the original view-space z value to be copied into the resulting w component. Therefore, after multiplying the vertex by the projection matrix, the clip-space w value becomes equal to the original z value.

This step will become important at perspective divide.

The resulting vector values will be:

$$x_{clip} = (f/a) * x$$

$$y_{clip} = f * y$$

$$z_{clip} = far/(far - near) * z - far * near / (far - near)$$

$$w_{clip} = z$$

The fourth row of the projection matrix copies the original view-space (z) value into the (w) component. This is important because the perspective divide later divides the clip-space coordinates by w_{clip} . This creates the perspective effect: objects that are

further from the camera, will appear smaller on screen.

After the projection matrix multiplication, the vertex is still not in screen space. It will be converted from clip space into normalized device coordinates by performing the perspective divide:

```

ndc_x = clip.x / clip.w
ndc_y = clip.y / clip.w

```

```
ndc_z = clip.z / clip.w
```

These equations come out to be:

$$x_{ndc} = (f/a) * (x/z)$$

$$y_{ndc} = f * (y/z)$$

This division is what creates the perspective effect. The bigger the Z becomes, the smaller the objects and (x_ndc , y_ndc).

The normalized depth value is calculated:

$$z_{ndc} = z_{clip} / w_{clip}$$

This maps the depth value into a normalized range. With this projection matrix, the near clipping plane maps to 0, and the far clipping plane maps to 1.

After the perspective divide, the vertex is in normalized device coordinates. The final step is to convert normalized device coordinates into screen-space pixel coordinates:

$$x_{screen} = ((x_{ndc} + 1) / 2 * width$$

$$y_{screen} = ((1 - y_{ndc}) / 2 * height$$

```
screen_x = (ndc_x + 1.0f0) * 0.5f0 * Float32(app.width)
screen_y = (1.0f0 - ndc_y) * 0.5f0 * Float32(app.height)
```

The y-coordinate uses (1 - y_ndc) because normalized device coordinates usually treat positive y as upward, while screen coordinates usually start at the top of the screen and increase downward.

After these steps, the renderer has the vertex in screen-space, and the triangle can be passed to the rasterizer.

4.7 Scanline Triangle Rasterization

After all the projection steps have been completed, the renderer moves to rasterizing each triangle. The vertices of the triangles are now in screen space, and the renderer knows the three projected points of the triangle.

The rasterizer in this renderer uses a scanline-based approach with DDA-style edge interpolation. The basic concept of this approach is to divide each triangle into two vertical regions, and process the triangle one row/line at a time. For each row, the rasterizer calculates the starting and ending points of the row, which lie on the edges of the triangle line. All pixels between those two lines are considered part of the triangle, and will be passed to the actual draw function that populates the ColorBuffer at those coordinates [29].

After this point, each vertex contains screen-space x and y coordinates, as well as depth-related values that are used later for interpolation and Z-buffering. The vertices are then sorted by their y-coordinate:

```
SwappedCoords, SwappedUVs = SwapCoords(ScreenVertices, FaceUVs)
```

Afterward, the sorted vertices will be named vec0, vec1, and vec2:

```
vec0 = SwappedCoords[1]
vec1 = SwappedCoords[2]
vec2 = SwappedCoords[3]
```

and further sectioned as:

```
x0 = vec0.x
y0 = vec0.y
x1 = vec1.x
y1 = vec1.y
x2 = vec2.x
y2 = vec2.y
```

where $y_0 \leq y_1 \leq y_2$

The UV values will be used and explained later, at the texture mapping stage.

Sorting the vertices this way ensures that during the calculation the triangle can be split into two regions, where the first region will span from y_0 to y_1 , and the second will span from y_1 to y_2

For an edge between two points (x_i, y_i) and (x_j, y_j) , the x-coordinate at a particular scanline y can be calculated using linear interpolation:

$$x_y = \frac{(y - y_i) * (x_j - x_i)}{y_j - y_i} + x_i$$

where the fraction:

$$(x_j - x_i) / (y_j - y_i)$$

is the inverse slope of the edge, and the scanline will use it to calculate how much the x coordinate changes when it moves one pixel in y (vertically) [29].

For the upper part of the triangle, the renderer interpolates between $\text{vec0} \rightarrow \text{v1}$ and $\text{vec0} \rightarrow \text{vec2}$ as such:

```
invSlope1 = (x1 - x0) / (y1 - y0)
invSlope2 = (x2 - x0) / (y2 - y0)
```

For each scanline, the two edge intersections are calculated as:

```
for y in yStart:yEnd
    fy = Float32(y)
    xStart = x0 + (fy - y0) * invSlope1
    xEnd = x0 + (fy - y0) * invSlope2
```

where yStart and yEnd are:

```
yStart = ceil(Int, y0)
yEnd = floor(Int, y1)
```

to convert these values into integer pixel coordinates.

xStart and xEnd represent the horizontal span of each line. Since it is not known whether xStart is on the left or right side, they might need to be swapped:

```
if xEnd < xStart
    xStart, xEnd = xEnd, xStart
end
```

These coordinates are also converted into integers from floating-point values:

```
ixStart = ceil(Int, xStart)
ixEnd = floor(Int, xEnd)
```

The use of ceil for the start and floor for the end ensures that only integer pixel positions inside the horizontal span are processed.

As a final step for the upper part, the renderer loops through each line and passes the pixels inside the triangle to the draw function:

```
for x in ixStart:ixEnd
    DrawColoredPixel(x, y, vec0, vec1, vec2, uv0, uv1, uv2,
        color, app)
end
```

For just colored draw, only the x and y coordinates are needed with the color value in the DrawColoredPixel function; the rest are used for barycentric weight calculations, Z-buffering, and interpolation, and will be looked at later.

The lower part of the triangle is processed similarly, except it uses the edges $\text{vec1} \rightarrow \text{vec2}$ and $\text{vec0} \rightarrow \text{vec2}$

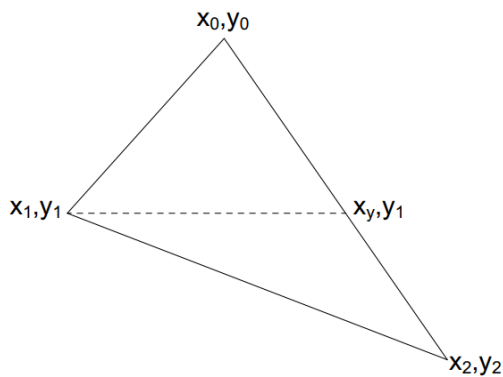


Figure 4.8

Figure 4.8 shows a possible division of a triangle using this method [29].

Limits of this implementation:

If two y values are equal, one of the triangle parts may have a height of 0, so to avoid later division by 0, the renderer has a guard before the calculations take place. For the upper loop:

```
if y1 - y0 != 0.0f0
```

and the lower loop:

```
if y2 - y1 != 0.0f0
```

Additionally, with this implementation, the middle line can be drawn twice, since the upper part ends at:

```
yEnd = floor(Int, y1)
```

And the lower part starts at:

```
yStart = ceil(Int, y1)
```

4.8 Texture Mapping

In 3D graphics, texture mapping is used to apply a 2D image onto the surface of a 3D object. Without this, the object can only be rendered using a solid color, or the color used from lighting calculations. Using texture mapping, each pixel on a triangle can be assigned a unique color from an image. With this, the renderer can display surface details, patterns, or material appearances without adding extra geometry [31].

In this renderer, textures are calculated with the use of UV coordinates obtained from the OBJ file. A UV coordinate takes the form of:

$uv = (u, v)$

```
struct Tex2
    u::Float32
    v::Float32
end
```

In each face, each vertex has a stored UV coordinate in the form of:

```
mutable struct Face
    a::Int
    b::Int
    c::Int

    a_uv::Tex2
    b_uv::Tex2
    c_uv::Tex2
end
```

For each vertex, the u coordinate represents the horizontal location (between 0 and 1), where 0 is the left side, and 1 is the right side of the texture. Similarly, the v coordinate represents the vertical limits of the texture [30].

After all the projection steps take place, the renderer operates with generated pixel coordinates. For each of the pixels inside the triangle, the renderer has to determine which pixel correlates to that from the texture. Since the only known UV coordinates are at the three vertices of the triangle, the rest has to be interpolated [20].

In this implementation, the renderer uses barycentric weights to determine the position of each pixel in the triangle. The three weights of a pixel will be written as:

α , β , γ

Where the three points of a triangle are:

A, B, C, and the point in the triangle that we want to find is: P

Shown in Figure 4.10.1

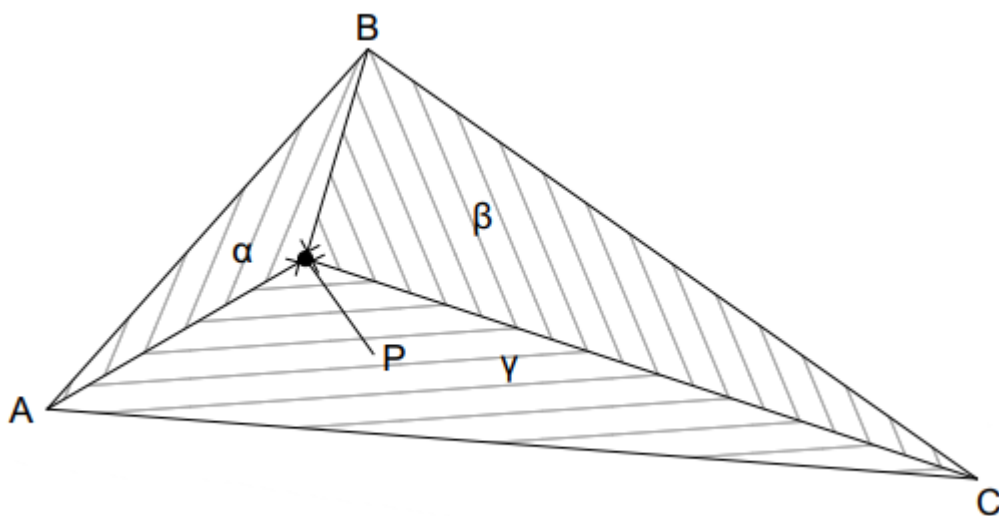


Figure 4.10.1

Then point P can be written as:

$$P = \alpha A + \beta B + \gamma C \quad \text{where} \quad \alpha + \beta + \gamma = 1$$

These values are not always positive. We can say that point P is inside the triangle with confidence if all three weights are positive [20].

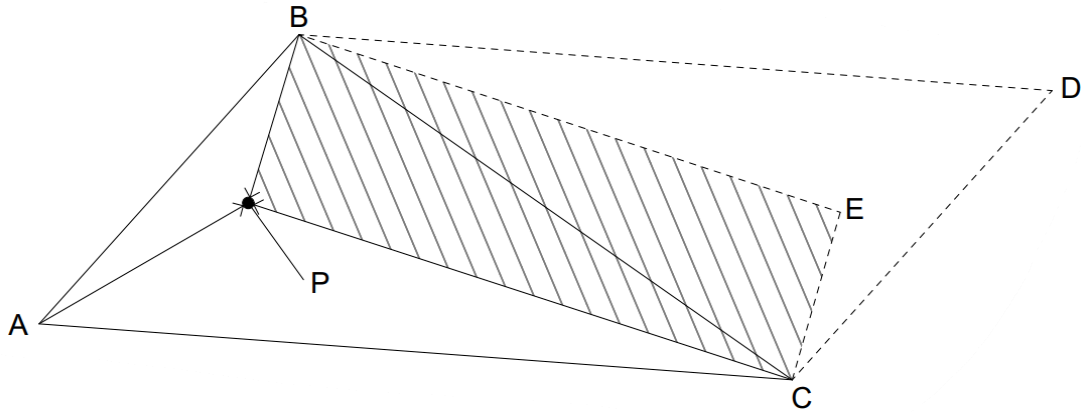


Figure 4.10.2

With the area of a triangle being A_t , and the total area of the ABC triangle A, we can compute the Barycentric coordinates as such:

$$\alpha = A_a / A$$

$$\beta = A_b / A$$

$$\gamma = A_c / A$$

For calculating these areas, the property of a cross product will be used, which says it is equal to the area of the parallelogram formed by the two vectors, so to get the area of the triangle, $\frac{1}{2}$ of it will be used [20]. But in this implementation, since both sides of the division use $\frac{1}{2}$, it will cancel out. So from Figure 4.10.2, the full calculation comes out to be:

$$\alpha = \frac{\|PC \times PB\|}{\|AC \times AB\|} \quad \beta = \frac{\|AC \times AP\|}{\|AC \times AB\|} \quad \gamma = 1 - \alpha - \beta$$

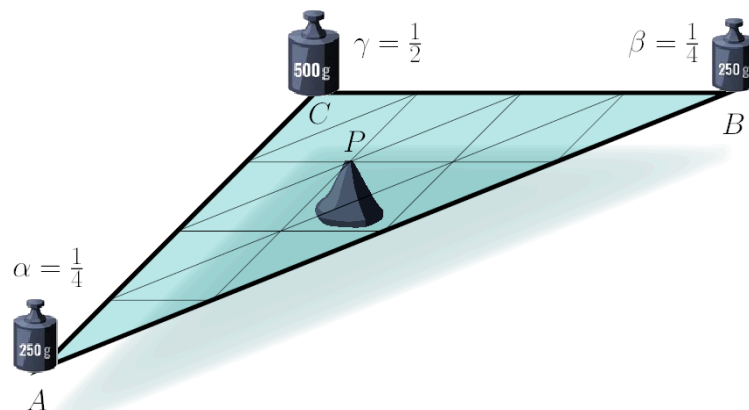


Figure 4.10.3 From [20]

Figure 4.10.3 illustrates how to imagine a point P on a triangle, with example-calculated weights.

In this implementation, the weights are calculated:

```
function BarycentricWeights(p::Vec2, a::Vec2, b::Vec2, c::Vec2)::Vec3
    denom = ((b.y - c.y) * (a.x - c.x) + (c.x - b.x) * (a.y - c.y))

    if denom == 0.0f0
        return Vec3(-1.0f0, -1.0f0, -1.0f0)
    end

    alpha::Float32 = ((b.y - c.y) * (p.x - c.x) + (c.x - b.x) * (p.y - c.y)) / denom
    beta::Float32 = ((c.y - a.y) * (p.x - c.x) + (a.x - c.x) * (p.y - c.y)) / denom
    gamma::Float32 = 1.0f0 - alpha - beta
    return Vec3(alpha, beta, gamma)
end
```

where denom is the area of the whole parallelogram, and the three weights are calculated based on the formula discussed above.

Finally, we can calculate the UV coordinates [20]:

$$u = \alpha * au + \beta * bu + \gamma * cu \quad v = \alpha * av + \beta * bv + \gamma * cv$$

However, these UV coordinates are not yet ready to calculate the actual pixel positions. After perspective projection, the coordinates are no longer linear in screen space [32] [33]. If these UV coordinates are used for interpolation, the texture might appear distorted .

To solve this, the renderer uses perspective-correct interpolation. Instead of directly interpolating u and v, it interpolates u / w, v / w, and 1 / w, where w is the homogeneous component produced by the projection stage. After this correction, the UV are reconstructed by dividing by the interpolated reciprocal of w [32] [33].

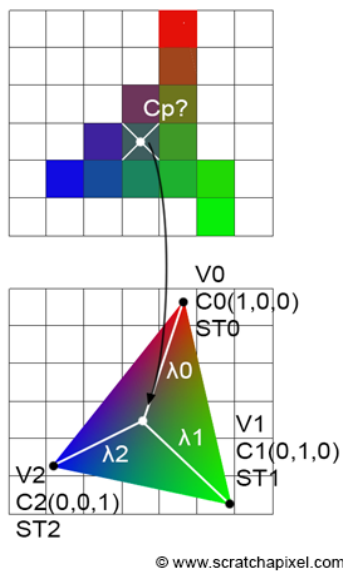


Figure 4.10.4 From [32]

Figure 4.10.4 depicts an example image of interpolating the colors at the triangle vertices, in order to find the value C_p using the calculated Barycentric weights.

In this implementation:

Function signature:

```
function DrawColoredPixel(x::Int, y::Int, a::Vec4, b::Vec4, c::Vec4,
    auv::Tex2, buv::Tex2, cuv::Tex2, color::UInt32, app::AppState)
```

Relevant implementation part:

```
au::Float32 = auv.u
av::Float32 = auv.v

bu::Float32 = buv.u
bv::Float32 = buv.v

cu::Float32 = cuv.u
cv::Float32 = cuv.v

av = 1 - av #line 10
bv = 1 - bv #line 11
cv = 1 - cv #line 12

Vec2Point::Vec2 = Vec2(Float32(x), Float32(y))
PointA::Vec2 = MakeVec2FromVec4(a.x, a.y, a.z, a.w)
PointB::Vec2 = MakeVec2FromVec4(b.x, b.y, b.z, b.w)
PointC::Vec2 = MakeVec2FromVec4(c.x, c.y, c.z, c.w)
Weights::Vec3 = BarycentricWeights(Vec2Point, PointA, PointB, PointC)
```

```

    InterpolatedU::Float32 = (au / a.w) * Weights.x + (bu / b.w) * Weights.y
+ (cu / c.w) * Weights.z
    InterpolatedV::Float32 = (av / a.w) * Weights.x + (bv / b.w) * Weights.y
+ (cv / c.w) * Weights.z
    IntepolatedReversedW::Float32 = (1 / a.w) * Weights.x + (1 / b.w) *
Weights.y + (1 / c.w) * Weights.z

    InterpolatedU /= IntepolatedReversedW
    InterpolatedV /= IntepolatedReversedW

```

In this code, a.w, b.w, and c.w are the w components of the projected triangle vertices. The points a, b, and c are the three vertices of the triangle, and auv, buv, and cuv are the texture uv coordinates of the same vertices.

The vertical texture coordinates are flipped on lines 10, 11, and 12 because texture coordinate systems and image coordinate systems often use different vertical directions. In many coordinate systems, v = 0 is considered the bottom of the texture, while image pixel coordinates often start at the top.

After this correction, the renderer does the z-buffer depth check, which will be discussed in the next chapter, and samples the texture:

```
baseColor = SampleTexture(app.Texture, InterpolatedU, InterpolatedV)
```

In Sample Texture:

```

function          SampleTexture(texture::Texture,          u::Float32,
v::Float32)::UInt32
    u = clamp(u, 0.0f0, 1.0f0)
    v = clamp(v, 0.0f0, 1.0f0)
    texX = floor(Int, u * Float32(texture.width - 1))
    texY = floor(Int, v * Float32(texture.height - 1))
    index = texY * texture.width + texX + 1
    return texture.pixels[index]
end

```

Returns the color at the passed UV coordinates, which will be later passed to the draw function:

```
DrawPixel!(app, x, y, baseColor)
```

4.9 Flat Shading

Flat shading is a popular and relatively simple shading model. Using this technique, each face of the mesh will have one color assigned to it. This color is calculated using the normal vector of a face. It is used to determine the face's orientation relative to the light source and to determine the intensity, or brightness, of the color [19].

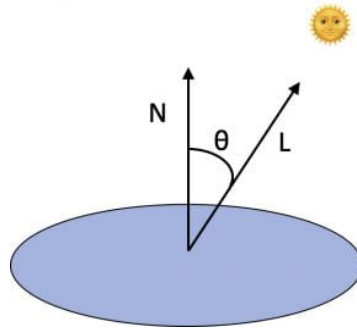


Figure 4.11, from [19]

In this renderer, flat shading is calculated before the triangle is rasterized. The function receives the three vertices of the triangle and a light direction:

```
function CalcLightIntensity(vertices::Vector{Vec3},  
                             LocalLight::Vec3)::Float32  
  
    NormalVec::Vec3 = Vec3Normal(vertices[1], vertices[2], #line4  
                                   vertices[3])  
  
    NormalizedVec::Vec3 = Normalize(NormalVec) #line7  
    NormalizedLightRay::Vec3 = Normalize(LocalLight) #line8  
    DotProd::Float32 = - Vec3Dot(NormalizedLightRay, #line9  
                                   NormalizedVec)  
  
    if (DotProd < 0) DotProd = 0 end  
    if (DotProd > 1) DotProd = 1 end  
  
    return DotProd  
end
```

Lines 4 and 7 calculate the triangle normal vector. The resulting vector is perpendicular to the triangle surface, and it indicates the face direction. The light direction is predefined, and it is normalized in the function.

The renderer then calculates the dot product between the normalized light direction and the normalized face normal on line 8.

A negative sign is used because of the direction convention of the light vector in the renderer. The light vector points in the opposite direction compared to the

normal-facing convention used in the lighting calculation. Negating the dot product makes triangles facing the light receive a positive intensity.

The calculated intensity is then applied to the color in ApplyLight:

```
function ApplyLight(originalColor::UInt32,
percentageFactor::Float32)::UInt32
    a = originalColor & 0xff000000 #line 3
    r = (originalColor >> 16) & 0xff #line 4
    g = (originalColor >> 8) & 0xff #line 5
    b = originalColor & 0xff #line 6

    r = UInt32(clamp(round(Int, Float32(r) * percentageFactor), #line 8
0, 255))
    g = UInt32(clamp(round(Int, Float32(g) * percentageFactor), #line 10
0, 255))
    b = UInt32(clamp(round(Int, Float32(b) * percentageFactor), #line 12
0, 255))

    return a | (r << 16) | (g << 8) | b
end
```

The final value is then clamped between 0 and 1, and passed back to the caller, which then applies the light on the given surface.

4.10 Z-Buffer and Fog

Before submitting the final color to the draw function, the renderer has to decide whether it is supposed to be rendered or not, based on its depth. This is because, if there are two pixels in the same position, but with varying depth, we only want to render the one in front. The renderer therefore needs a method to decide which triangle is closest to the camera at each pixel. This is handled using a Z-buffer [34].

The Z-Buffer, in general, is a depth buffer one dimensionally array that has one depth value for each pixel of the ColorBuffer. It stores the closest depth value found so far at that location. When a new pixel is about to be sent to draw, it first checks if there are any pixels with a closer depth value. If so, the pixel doesn't get rendered, but if it is the closest pixel at that location, it gets sent to be drawn, and the Z-Buffer is updated at that pixel position [34][35].

As discussed earlier, in this implementation, the renderer interpolates the reciprocal of the homogeneous w component.

Afterward, the depth value is reconstructed:

```
Depth::Float32 = 1.0f0 / IntepolatedReversedW
```

Since the projected triangle is already in screen space, directly interpolating depth values can produce incorrect results under perspective projection, so this approach gives a better approximation to the original perspective relationship.

The screen coordinates x,y are then converted into an index in the array:

```
Index = app.width * y + x + 1
if Depth < app.zBuffer[Index]

    baseColor::UInt32 = color

    if CurrDisplayState == TEXTURE || CurrDisplayState ==
        TEXTURE_WIREFRAME
        baseColor = SampleTexture(app.Texture, InterpolatedU,
            InterpolatedV)
    end
    DrawPixel!(app, x, y, baseColor)
    app.zBuffer[Index] = Depth

end
```

Finally, the renderer checks if the user wants a textured output; if so, it samples and sends to draw; otherwise, it just sends the original color to the draw function with no texture, and updates the depth value at that pixel position.

Fog is applied after the visible pixel color has been determined. While the Z-buffer decides whether a pixel should be drawn, fog changes the final appearance of the pixel based on distance.

A simple linear fog model can be written as [36] [37]:

$$\text{fogFactor} = \text{clamp}((\text{fogEnd} - \text{distance}) / (\text{fogEnd} - \text{fogStart}), 0, 1)$$

In the renderer:

```
function ApplyFogToWholeScreen!(app::AppState)
    for y in 0:app.height-1
        for x in 0:app.width-1
            index = y * app.width + x + 1

            if app.zBuffer[index] == Inf32
                app.ColorBuffer[index] = FOG_COLOR
            else
                depth = app.zBuffer[index]
                app.ColorBuffer[index] =
                    ApplyFog(app.ColorBuffer[index], depth)
            end
        end
    end
end
```

```
        end
    end
end
end
```

Where the renderer loops through the indexes of the two buffers and checks whether the Z-Buffer has a value written in the given index. If not, that means the renderer decided earlier that there should be nothing; thus, it makes the color of that index the FOG_COLOR (gray), and if there is a drawn pixel with a depth value, it applies the fog effect as such:

```
function ApplyFog(color::UInt32, distance::Float32)::UInt32
    fogAmount = (distance - FOG_START) / (FOG_END - FOG_START)
    fogAmount = clamp(fogAmount, 0.0f0, 1.0f0)

    return LerpColorARGB(color, FOG_COLOR, fogAmount)
end
```

With the help of a lerp function, it returns the final color.

This means that closer pixels keep more of their original color, while farther pixels are blended more strongly with the fog color [36] [37].

Using this approach, the application of the fog effect becomes a post-process calculation.

5. Evaluation

The renderer was evaluated according to the features defined in the project scope. Each rendering step was tested separately where possible, and observed as a part of the complete rendering pipeline. This evaluation is more qualitative than performative due to the nature of the thesis project.

The visual output of different rendering modes were also inspected. The wireframe mode was useful, to determine whether the OBJ files were loaded correctly, and they preserved their original structure.

The Z-Buffer was tested using overlapping geometry, and it proved to be correct. Backface culling was tested using the same geometry, and was switched on and off during rendering.

The textured rendering was tested by loading different textures, and rendering them. It was also switched on and off, along with the fill rendering mode.

6. Conclusion

This thesis presented the design and implementation of a CPU-based 3D software renderer written in Julia. The goal of the project was to explore how a 3D model can be transformed from geometric data into a final 2D image.

The renderer successfully implemented several code stages of a 3D rendering pipeline, and these together make a good subset of a basic OpenGL 1.0 implementation. It can load OBJ files, textures, it can represent meshes internally, transform the vertices through the several necessary matrices, can move the camera through the scene, performs backface culling, rasterizes the triangles using scanline-based approach, tests visibility using the Z-Buffer, applies perspective-correct interpolation, calculates flat shading, and finally applies depth based fog effect.

The main research question of this thesis was that how to implement and explain the main stages of a general pipeline using math and Julia implementation. The implementation shows that these stages can be built manually, in a structured way, following the logical path of the pipeline itself. Each of these stages depend on the previous one, and together they explain how a 3D pipeline comes together.

The most notable result of this project is that, it becomes much easier to understand concepts like what the view matrix is, or how projection works, when it is paired up with direct implementation. The renderer has several limitation however, as it only uses a simple depth based fog, it only implements flat shading, doesn't use advanced texture filtering, skeletal meshes, and doesn't implement shadows.

Overall, this thesis depicts that it is an effective way to understand the inner working of a rendering pipeline to build it from scratch with no graphics APIs. This project does not aim to replace any existing system, since it does not come near to modern graphics APIs in complexity, or in the number of features, however it is a fun way to learn the basics of them.

References

- [1] E. Potter, "Building a Software Render Engine from Scratch," Elijah Potter, Jan. 31, 2024. [Online]. Available: <https://elijahpotter.dev/articles/building-a-software-render-engine-from-scratch>.
- [2] Graphics Compendium, "Overview," Graphics Compendium. [Online]. Available: <https://graphicscompendium.com/intro/01-graphics-pipeline>.
- [3] Khronos Group, "Rendering Pipeline Overview," OpenGL Wiki. [Online]. Available: https://wikis.khronos.org/opengl/Rendering_Pipeline_Overview.
- [4] Wikibooks contributors, "Cg Programming/Vertex Transformations," Wikibooks. [Online]. Available: https://en.wikibooks.org/wiki/Cg_Programming/Vertex_Transformations.
- [5] MDN Web Docs, "WebGL model view projection," Mozilla Developer Network. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/WebGL_model_view_projection.
- [6] Scratchapixel, "An Overview of the Rasterization Algorithm," Scratchapixel. [Online]. Available: <https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/overview-rasterization-algorithm.html>.
- [7] "L-29145117-d9856facc5.pdf," e-bookshelf.de. [Online]. Available: <https://content.e-bookshelf.de/media/reading/L-29145117-d9856facc5.pdf>.
- [8] F. Dunn and I. Parberry, "Multiple Coordinate Spaces," Game Math. [Online]. Available: <https://gamemath.com/book/multiplespaces.html>
- [9] Friedrich Schiller University Jena, "Coordinate Systems," Computer Graphics lecture slides. [Online]. Available: https://vis.uni-jena.de/Lecture/ComputerGraphics/Lec6_CoordinateSystems.pdf.
- [10] J. de Vries, "Coordinate Systems," LearnOpenGL. [Online]. Available: <https://learnopengl.com/Getting-started/Coordinate-Systems>.

- [11] C. Cai, "CS532 3D Computer Vision, Week 10," Stevens Institute of Technology. [Online]. Available: https://www.changjiangcai.com/files/courses-slides/CS532-3D-CV-Stevens/cs532f16_Week10.pdf.
- [12] Princeton University, "14. Mesh," COS 526 Lecture Notes, Fall 2016. [Online]. Available: <https://www.cs.princeton.edu/courses/archive/fall16/cos526/lectures/14-mesh.pdf>.
- [13] Scratchapixel, "Shading Normals," Scratchapixel. [Online]. Available: <https://www.scratchapixel.com/lessons/3d-basic-rendering/introduction-to-shading/shading-normals.html>.
- [14] Scratchapixel, "The OBJ File Format," Scratchapixel. [Online]. Available: <https://www.scratchapixel.com/lessons/3d-basic-rendering/obj-file-format/obj-file-format.html>.
- [15] M. Segal and K. Akeley, "The OpenGL Graphics System: A Specification, Version 1.0," Silicon Graphics, Inc., 1992. [Online]. Available: <https://registry.khronos.org/OpenGL/specs/gl/glspec10.pdf>.
- [16] Microsoft, "Rasterization Rules (Direct3D 9)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/direct3d9/rasterization-rules>.
- [17] T. Thormählen, "3D Transformations," Graphics Programming, Part 5, Chapter 2, Nov. 4, 2022. [Online]. Available: https://www.mathematik.uni-marburg.de/~thormae/lectures/graphics1/graphics_5_2_eng_web.html#3.
- [18] J. de Vries, "Face culling," LearnOpenGL. [Online]. Available: <https://learnopengl.com/Advanced-OpenGL/Face-culling>
- [19] M. J. Laszlo, "Flat and Smooth Shading," Nova Southeastern University. [Online]. Available: <https://sites.nova.edu/mjl/graphics/lighting/flat-and-smooth-shading/>.
- [20] Haqr, "Barycentric Coordinates," Tiny Renderer. [Online]. Available: <https://haqr.eu/tinyrendererer/barycentric/>
- [21] Graphics Compendium, "Barycentric Coordinates," Graphics Compendium. [Online]. Available: <https://graphicscompendium.com/intro/03-barycentric-coordinates>.
- [22] Carnegie Mellon University, "Camera Matrix," 16-385 Computer Vision lecture slides, Spring 2017. [Online]. Available: https://www.cs.cmu.edu/~16385/s17/Slides/11.1_Camera_matrix.pdf
- [23] Coding Labs, "World, View and Projection Transformation Matrices," Coding Labs. [Online]. Available: https://www.codinglabs.net/article_world_view_projection_matrix.aspx.

- [24] Newcastle University, "Tutorial 2 Part A: Vertex Transformation," Graphics for Games. [Online]. Available: <https://research.ncl.ac.uk/game/mastersdegree/graphicsforgames/vertextransformation/Tutorial%20-%20Vertex%20Transformation.pdf>.
- [25] OpenStax, "8.8 Vectors," Mathematics LibreTexts. [Online]. Available: https://math.libretexts.org/Bookshelves/Precalculus/Precalculus_1e_%28OpenStax%29/08%3A_Further_Applications_of_Trigonometry/8.08%3A_Vectors.
- [26] S. H. Ahn, "OpenGL Camera," Song Ho OpenGL Tutorials. [Online]. Available: https://www.songho.ca/opengl/gl_camera.html.
- [27] A. Danner, "Week 5: LookAt/View and Projection Matrices," Swarthmore College. [Online]. Available: <https://www.cs.swarthmore.edu/~adanner/cs40/f25/notes/week05.html>.
- [28] Scratchapixel, "Single vs Double-Sided Triangle and Backface Culling," Scratchapixel. [Online]. Available: <https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-rendering-a-triangle/single-vs-double-sided-triangle-backface-culling.html>.
- [29] Sunshine2k, "Software Rasterization Algorithms for Filling Triangles," Sunshine2k. [Online]. Available: <https://www.sunshine2k.de/coding/java/TriangleRasterization/TriangleRasterization.html>.
- [30] Rose-Hulman Institute of Technology, "Textures," CSSE351 Computer Graphics. [Online]. Available: <https://www.rose-hulman.edu/class/cs/csse351-online/m5/Textures/>.
- [31] J. Bell, "Texture Mapping," University of Illinois Chicago Computer Graphics Course Notes. [Online]. Available: <https://www.cs.uic.edu/~jbell/CourseNotes/ComputerGraphics/TextureMapping.html>.
- [32] K. Bala, "Depth Interpolation and Perspective Correct Texturing," Cornell University, CS 4620 Lecture Notes, 2015. [Online]. Available: <https://www.cs.cornell.edu/courses/cs4620/2015fa/lectures/PerspectiveCorrectZU.pdf>.
- [33] Scratchapixel, "Perspective Correct Interpolation and Vertex Attributes," Scratchapixel. [Online]. Available: <https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/perspective-correct-interpolation-vertex-attributes.html>
- [34] Scratchapixel, "The Visibility Problem, the Depth Buffer, and Depth Interpolation," Scratchapixel. [Online]. Available: <https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/visibility-problem-depth-buffer-depth-interpolation.html>
- [35] J. de Vries, "Depth testing," LearnOpenGL. [Online]. Available: <https://learnopengl.com/Advanced-OpenGL/Depth-testing>.

[36] Microsoft, "Fog Formulas (Direct3D 9)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/direct3d9/fog-formulas>

[37] OpenGL Notes, "Blending, Aliasing, and Fog," Graphics with OpenGL 0.1 documentation. [Online]. Available: <https://opengl-notes.readthedocs.io/en/latest/topics/texturing/aliasing.html>.

[38] <https://drive.google.com/file/d/12kHv7RI57MU0hCGXvjQpK1VmJ0yuoDPR/view?usp=sharing>